

25CS311

Object-Oriented Programming

String Handling



Dr. J. Anitha
Professor - CSE

Learning & Practicum Outcomes



Learning Unit Outcome

Apply appropriate string functions and generate dynamic string templates for the given text processing application.



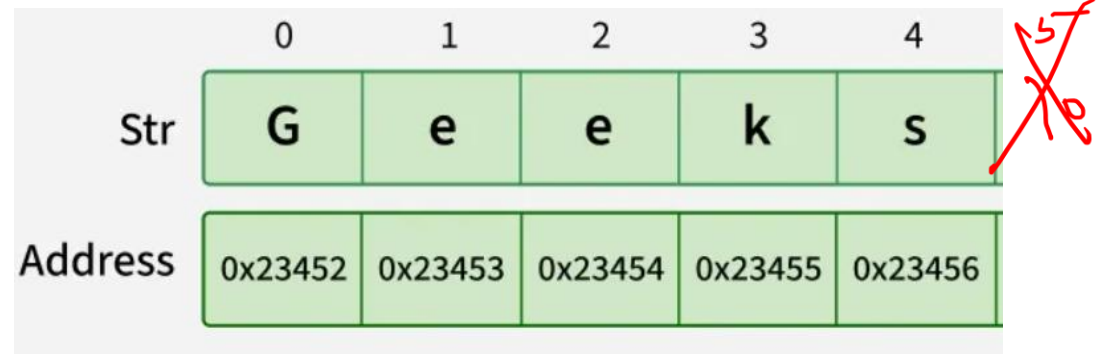
Practicum Outcome

Choose relevant string-handling techniques for developing the given text-processing application.

String

- A String is a sequence of characters.
- Three ways to create String in Java
 - ② • String ✓
 - StringBuffer ✓
 - StringBuilder ✓
- The String, StringBuffer, and StringBuilder classes are defined in java.lang.

String Str = "Geeks";



```
String s1 = "welcome";  
String s2 = "welcome";
```

```
String s3 = new String("welcome");
```

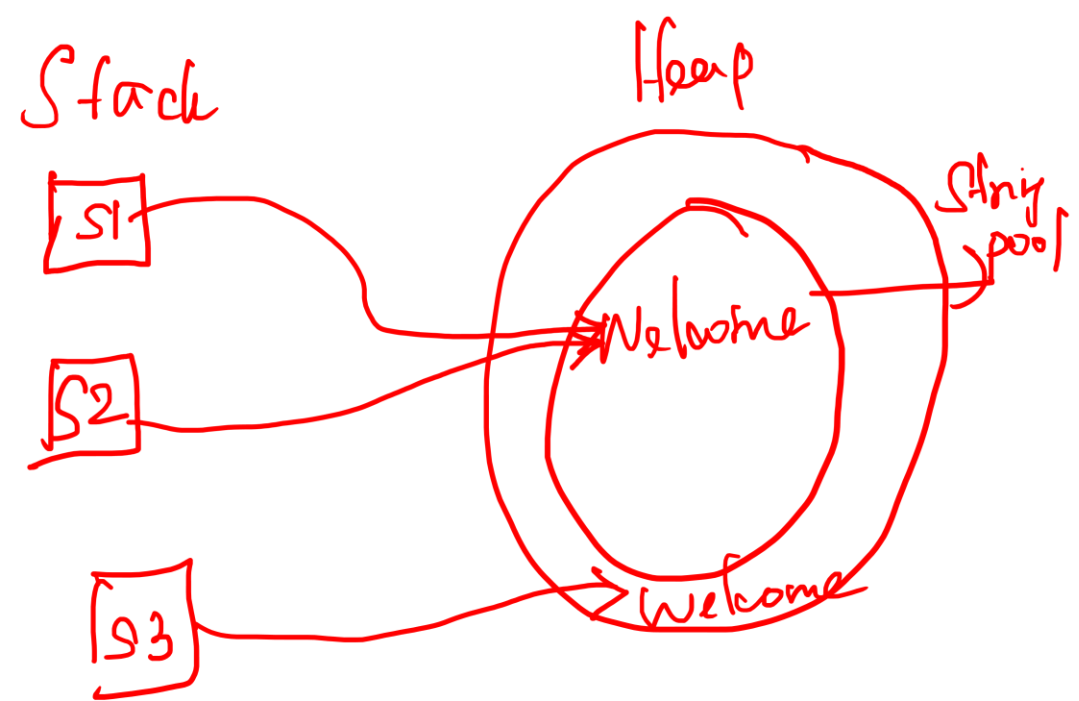
```
if(s1.equals(s2))  
    true
```

```
if(s1.equals(s3))  
    true
```

equals $\rightarrow$ Content
== $\rightarrow$ reference

```
if(s1 == s2)  
    true
```

```
if(s1 == s3)  
    false
```



Immutable

- cannot be changed

Mutable

- can be changed

```
String s=new String  
("Hello"); s+="Hi";
```

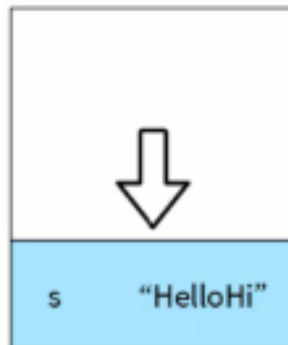
String



Memory

```
StringBuffer s=new  
StringBuffer("Hello");  
s.append("Hi");
```

StringBuffer



Memory

String

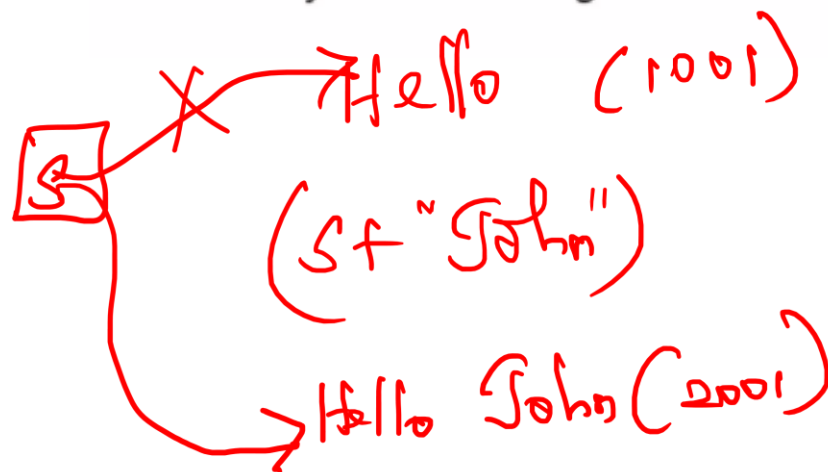
StringBuilder
StringBuffer

String s = "Hello";

s = s + "John";

StringBuffer sb = new StringBuffer("Hi");

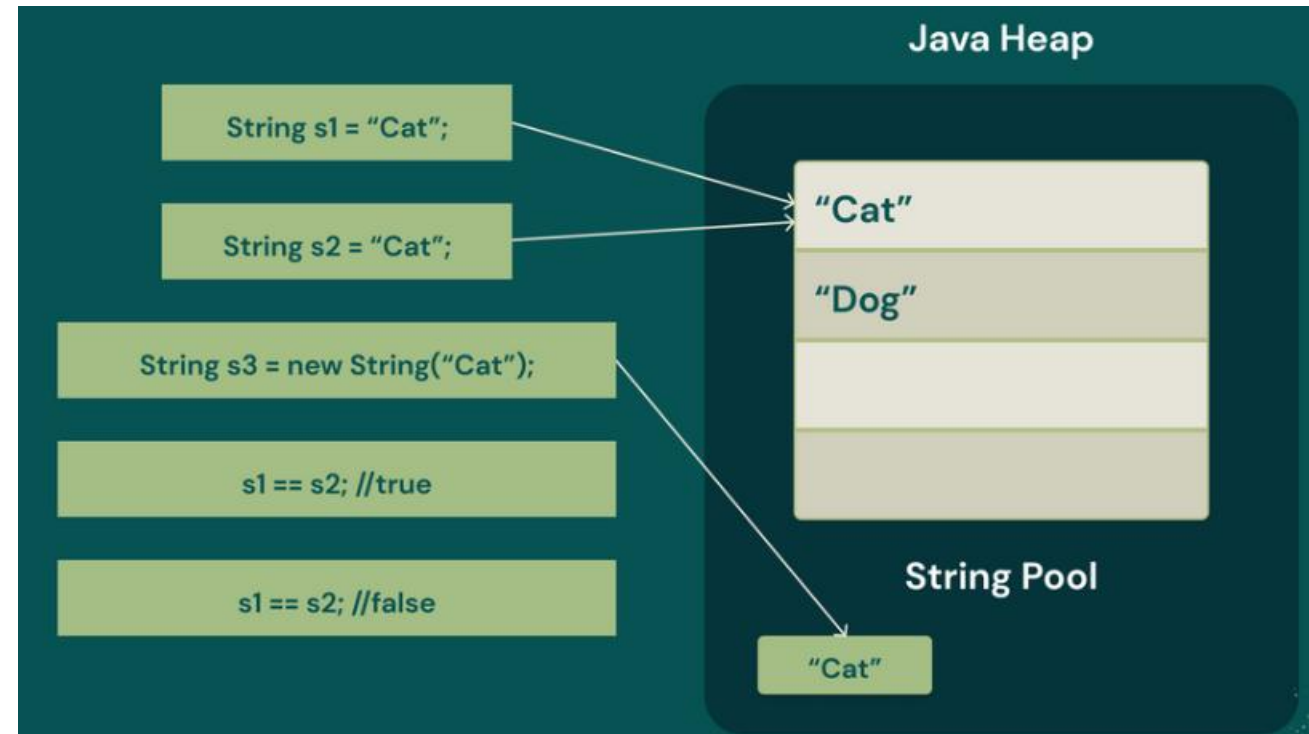
sb.concat("John")



Feature	`String`	`StringBuffer`	`StringBuilder`
Mutability	Immutable	Mutable	Mutable
Thread-Safety	Thread-Safe	Thread-Safe (synchronized methods)	Not Thread-Safe (non-synchronized)
Performance	Lower performance in mutable operations due to object creation	Higher performance in mutable operations due to mutable nature	Higher performance in mutable operations due to non-synchronization
Memory Area	Heap Memory (String Pool)	Heap Memory (Internal mutable array)	Heap Memory (Internal mutable array)
Use Case	When string is constant or infrequent modifications	When string is modified in multi-threaded environment	When string is modified in single-threaded environment

Creating a String

- Using String literal
- Using 'new' keyword



Feature	String Literal
Syntax	<code>String s = "Java";</code>
Memory Location	String Constant Pool
Object Creation	Reuses existing object if available
Memory Usage	Efficient
Performance	Faster
Duplicate Objects	Avoided

new Keyword
<code>String s = new String("Java");</code>
New object in Heap
Always creates a new object
More memory
Slightly slower
Created every time

String Constructors

→ object using new keyword

Empty string

```
String s = new String();
```

From literal text

```
String s = new String("welcome");
```

From a char array

```
char carr[] = {'a','b','c'};  
String s = new String(carr);
```

"abc"

From a sub-range of a char array
(chars, startIndex, numChars)

```
char c[] = {'a','b','c','d','e','f'};  
String s = new String(c, 2, 3);
```

"cde"

From another String object

```
String s1 = new String("Java");  
String s2 = new String(s1);
```

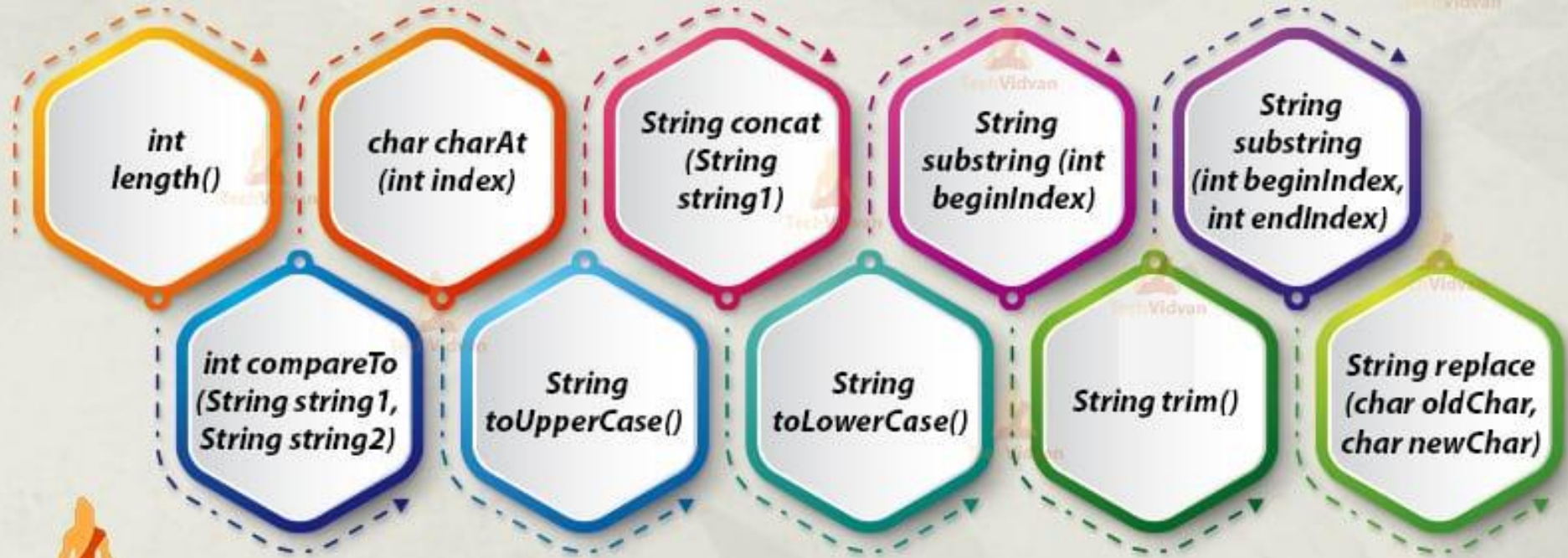
"Java" "Java"

From a byte array (ASCII)
String(byte[], start, numChars)

```
byte a[]={65,66,67,68,69,70}; → ASCII  
String s1=new String(a);  
String s2=new String(a,2,3);
```

"ABCDEF" "CDE"

METHODS OF JAVA STRING



Predict the output

- String s=4+4⁸+"Hello";

8Hello

+ → left to right

- String s="Hello"+^{String}4+4;

Hello44

- String s="Hello"+(4+4);

Hello 8

Activity

A company's HR system wants to analyze employee names for linguistic statistics. Develop a program that accepts an employee's full name and counts the total number of vowels present in it

```
System.out.print("Enter a string: ");
String str = sc.nextLine();

// Convert the entire string to lowercase
str = str.toLowerCase();

int count = 0;

for (int i = 0; i < str.length(); i++) {

    char ch = str.charAt(i);

    if (ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u')
        count++;
}

System.out.println("Number of vowels = " + count);
```

String Length

DEFINITION

`length()` returns the number of characters currently stored in a `String` object. A `String` literal (text in double quotes) is automatically an instance of `String` — Java creates the object for you.

SYNTAX

```
int length()
```

`String st = new String("abcd");` is the same as writing `String st = "abcd";`

len

CODE SNIPPET

```
String s = new String("welcome");  
System.out.println(s.length());  
  
// also valid directly on a literal:  
int l = "welcome".length();
```

OUTPUT

7

String Concatenation

DEFINITION

The + operator joins Strings and values of other types into a single new String. Because Strings are immutable, concatenation always produces a brand-new String object.

SYNTAX

```
String s = "text " + variable + " more text";
```

Benefit: splitting one long literal across several + operators keeps source code readable without changing the resulting text.

CODE SNIPPET

```
int age = 10;  
String s = "He is " + age + " years old.";  
System.out.println(s);
```

```
String longStr = "This could have been " +  
    "a very long line that would have " +  
    "wrapped around.";
```

OUTPUT

```
He is 10 years old.
```



Character Extraction

DEFINITION

charAt() returns a single character at a given index.
getChars() copies a range of characters into a character array supplied by the caller.

SYNTAX

```
char charAt(int where)
void getChars(int srcStart, int srcEnd,
              char target[], int targetStart)
```

Substring copied by getChars spans [sourceStart, sourceEnd - 1].

chr

CODE SNIPPET

```
char ch = "abc".charAt(1);
String s = "This is a demo of the getChars method.";
char buf[] = new char[100];
s.getChars(10, 14, buf, 0);
System.out.println(buf);
```

Handwritten annotations:
 - Above "abc": 0 1 2
 - Above "demo": 0 1 2 3 4 5 6 7 8 9 10 11 12 13
 - Below 10, 14: 14 - 1 = 13

OUTPUT

ch = b

buf = "demo"

getBytes() and toCharArray()

DEFINITION

getBytes() converts a String into a byte array using the default ASCII/8-bit character-to-byte encoding.

toCharArray() converts every character of a String into a char array.

SYNTAX

```
byte[] getBytes()  
char[] toCharArray()
```

CODE SNIPPET

```
String str = "abcdef";  
  
byte b[] = str.getBytes();  
for (byte bt : b)  
    System.out.print(bt + " ");  
  
char c[] = str.toCharArray();  
for (char cr : c)  
    System.out.print(cr);
```

Handwritten notes:
s = "a b c d e f"
c = [a] [b] [c] [d] [e] [f]
0 1 2 3

OUTPUT

```
97 98 99 100 101 102  
abcdef
```

String Comparison — equals() – Check contents

==?

DEFINITION

`equals()` compares two Strings character-by-character and is case-sensitive. `equalsIgnoreCase()` performs the same comparison but ignores upper/lower case differences.

SYNTAX

```
boolean equals(Object str)
boolean equalsIgnoreCase(String str)
```

CODE SNIPPET

```
String s1="Hello", s2="Hello";
String s3="Good-bye", s4="hello";

System.out.println(s1.equals(s2)); ✓
System.out.println(s1.equals(s3));
System.out.println(s1.equals(s4));
System.out.println(s1.equalsIgnoreCase(s4));
```

OUTPUT

```
true
false
false
true
```

== vs equals()

DEFINITION

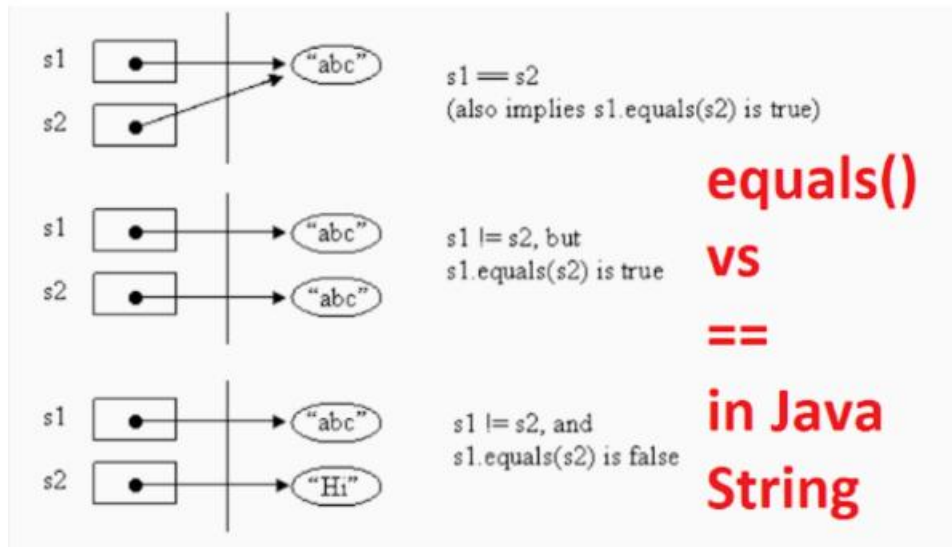
equals() compares character content;

== compares object references — whether two variables point to the exact same object in memory.

SYNTAX

str1 == str2

equals(): compares characters ==: compares references



CODE SNIPPET

```
String s1 = new String("Hello");
String s2 = s1;
String s3 = new String(s1);
System.out.println(s1.equals(s2));
System.out.println(s1 == s2);
System.out.println(s1.equals(s3));
System.out.println(s1 == s3);
```

OUTPUT

```
true
true
true
false
```

startsWith(), endsWith()

DEFINITION

startsWith()/endsWith() test whether a String begins or ends with a given substring.

SYNTAX

```
boolean startsWith(String str)
boolean endsWith(String str)
boolean startsWith(String str,int startIndex)
```

CODE SNIPPET

```
boolean b1="welcome".startsWith("we");
boolean b2="welcome".endsWith("me");
boolean b3="welcome".startsWith("lc",2);
```

OUTPUT

```
b1,b2,b3 = true true true
```

STRING COMPARISON

compareTo() & Sorting Strings

DEFINITION

compareTo() compares two Strings in natural (alphabetical / ASCII) order and returns a signed integer.

int

Return value	Meaning
< 0	Invoking string is less than str
> 0	Invoking string is greater than str
0	The <u>two strings</u> are equal ✓

```
int compareTo(String str)
int compareToIgnoreCase(String str)
```

s1.compareTo(s2)

"hello".compareTo("hello")

"h".compareTo("s")

h < s ⇒ -ve

"s".compareTo("h")
s > h ⇒ +ve

Array.Sort () ✓

<=>

CODE SNIPPET — Bubble sort with compareTo()

```
String str[] = {"Now","is","the","time",
               "for","all","good","men"};
for (int i=0;i<str.length;i++){
    for (int j=i+1;j<str.length;j++){
        if (str[i].compareTo(str[j]) > 0) {
            String t=str[j];
            str[j]=str[i];
            str[i]=t;
        }
    }
}
```

a[i] > a[i+1]

OUTPUT (alphabetical order)

Now, all, for, good, is, men, the, time

10 11
a[i] > a[j]

c d 0, +ve, -ve
s[i] < s[i+1]
s[i].compareTo(s[i+1])

ACTIVITY 1

Quick Check — Predict the Output

1	2	3	4	5	6	7	8	9	10	11	12	13
A	B	C	D	E	F	G	H	I	J	K	L	M
Z	Y	X	W	V	U	T	S	R	Q	P	O	N
26	25	24	23	22	21	20	19	18	17	16	15	14

```
String s1="hello";
String s2="hello";
String s3="meklo";
String s4="hemlo";
String s5="flag";
System.out.println(s1.compareTo(s2));
System.out.println(s1.compareTo(s3));
System.out.println(s1.compareTo(s4));
System.out.println(s1.compareTo(s5));
```

hello hello => 0
meklo
hemlo
flag

h < m = -5
l < m = -1
h > f = +2

```
String s1="hello";
String s2="";
String s3="me";
System.out.println(s1.compareTo(s2));
System.out.println(s2.compareTo(s3));
```

length of the str

hello > null +5
null < me -2

Return value	Meaning
< 0	Invoking string is less than str
> 0	Invoking string is greater than str
0	The two strings are equal

Searching Strings — indexOf() & lastIndexOf()

DEFINITION

indexOf() finds the first occurrence of a character or substring;
lastIndexOf() finds the last occurrence. If nothing matches, both return -1.

SYNTAX

```
int indexOf(char ch)  
int lastIndexOf(char ch)  
int indexOf(String str)  
int lastIndexOf(String str)
```

first occurrence

CODE SNIPPET

```
String str = "abcabdef";  
System.out.println(str.indexOf('a'));  
System.out.println(str.lastIndexOf('a'));  
  
String str2 = "abcabdefabcswabd";  
System.out.println(str2.indexOf("abd"));  
  
String str3 = "abcabdefabcswd";  
System.out.println(str3.lastIndexOf("abc"));
```

OUTPUT

0 3 3 8

Searching from a Starting Point

DEFINITION

Overloaded versions of `indexOf()`/`lastIndexOf()` accept a `startIndex`, letting the search begin partway through the `String` instead of at the very start or end.

SYNTAX

`int indexOf(char ch, int startIndex)`
`int lastIndexOf(char ch, int startIndex)`
`int indexOf(String str, int startIndex)`
`int lastIndexOf(String str, int startIndex)`

SEARCH DIRECTION

`indexOf()`: `startIndex` → end of string
`lastIndexOf()`: `startIndex` → index 0

EXAMPLE

`String str = "abcabdef";`

`str.indexOf('a', 1) → 3`

`str.lastIndexOf('a', 2) → 0`

Handwritten notes:
1 2 3 4 5 6 7
↑ ↑
= 0
`str.indexOf('a') ⇒ 0`
`str.lastIndexOf('a') ⇒ 3`

Modifying a String

substring(startIndex)

substring(startIndex, endIndex)

→ n-1



concat(String str)

replace(char original, char replacement)

trim() — removes leading/trailing whitespace

Use case

Username	10781
password	

s = "_781"

781



```
String str = "abcdefghij";
str.substring(2)
str.substring(2, 6)
```

```
String s1 = "one";
s1.concat("two")
```

```
"Hello".replace('l', 'w')
```

```
" Hello World ".trim()
```

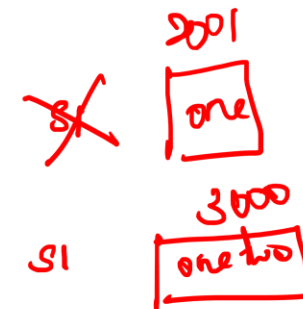
"cdefghij"

"cdef"

"onetwo"

"Hewwo"

"Hello World"



Quick Check — Predict the Output

What does this print?

```
if ("String".replace('t','T') ==  
    "String".replace('t','T'))  
    System.out.println("Equal");  
else  
    System.out.println("Not Equal");
```

"String".replace('t','T')

==

"String".replace('t','T')

2000 String → replace

2100 String

==

2000 String → replace
Not Equal

3000 String

Quick Check — Predict the Output

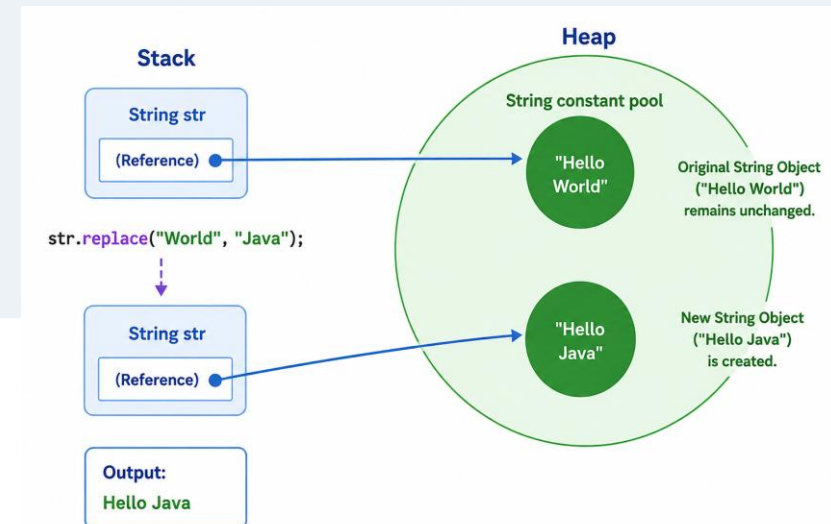
What does this print?

```
if ("String".replace('t','T') ==  
    "String".replace('t','T'))  
    System.out.println("Equal");  
else  
    System.out.println("Not Equal");
```

Answer & Why

Output: "Not Equal"

Each call to `replace()` returns a new `String` object in memory. `==` checks whether two references point to the same object — not whether their contents match — so two separately-created `"String"` results are never `==` equal, even though `.equals()` would say true.



Changing the Case of Characters

DEFINITION

`toUpperCase()` and `toLowerCase()` return a new `String` with every letter converted, leaving the original `String` unchanged (Strings are immutable).

SYNTAX

`String toLowerCase()` ✓

`String toUpperCase()` ✓

CODE SNIPPET

```
String str = "AbcDefgHij";  
System.out.println(str.toUpperCase());  
System.out.println(str.toLowerCase());
```

OUTPUT

ABCDEFGHIJ

abcdefghij



split(), replaceFirst() & replaceAll()

DEFINITION

split() breaks a String into an array wherever the regular expression matches.

replaceFirst()/replaceAll() substitute the first — or every — match of a regular expression with a new String.

char sarr = s1.toCharArray();
 wc = 1;
 for (int i = 0; i < sarr.length; i++)
 if (sarr[i] == ' ')
 wc++;

SYNTAX

```
String[] split(String regExp)
String[] split(String regExp, int max)
String replaceFirst(String regExp, String newStr)
String replaceAll(String regExp, String newStr)
```

CODE SNIPPET

```
String s1 = "This\\is\\a\\sample\\program";
String sarr[] = s1.split(" ");           // {"This","is","a","sample","program"}
String pair[] = s1.split(" ", 2);       // {"This","is a sample program"}

String s2 = "This is a sample pgm saved in filesample.java";
System.out.println(s2.replaceFirst("sam", "SAM")); // This is a SAMple pgm ...
System.out.println(s2.replaceAll("sam", "SAM"));   // ...SAMple pgm saved in fileSAMple.java
```

wc = sarr.length => 5

wc = 1;
 for (int i = 0; i < s1.length; i++)
 if (s1.charAt(i) == ' ')
 wc++;

valueOf() & toString()

DEFINITION

When a non-String value meets +, Java converts it using valueOf().

For primitive types, valueOf() formats the value; for objects, valueOf() calls the object's toString() method.

Evaluation is strictly left to right, so parentheses change the result.

SYNTAX / RULE

Primitive → String.valueOf(value)

Object → object.toString()

You may override toString() in your own class to control how it prints.

CODE SNIPPET

```
int i = 13;  
String s = String.valueOf(i);
```

```
String s1 = "Hello " + i + 5;
```

```
class A {  
    public String toString() { return "hai"; }  
}
```

```
A a = new A();
```

```
String s2 = "welcome " + a; // calls a.toString()
```

OUTPUT

```
s = 13
```

```
s1 = Hello 135
```

```
s2 = welcome A@2a139a55 //without a.toString()
```

```
s2 = welcome hai
```

StringBuffer — Introduction & Constructors

sb

DEFINITION

StringBuffer represents a growable and writable (mutable) sequence of characters.

Extra capacity is preallocated beyond what is currently used, to allow the buffer to grow.

All its methods are implicitly synchronized, making it thread-safe.

SYNTAX — CONSTRUCTORS

```
StringBuffer()           // buffer size 16
StringBuffer(int size)
StringBuffer(String str)
```

CODE SNIPPET

```
StringBuffer s1 = new StringBuffer(25);
StringBuffer s2 = new StringBuffer("welcome");

System.out.println(s1.capacity());
System.out.println(s2.capacity());
```

OUTPUT

```
25
23
```



Managing Buffer Size

SYNTAX

```
int length()           current length
int capacity()         length + spare room
void ensureCapacity(int min)
void setLength(int len)
```

Capacity growth rule for `ensureCapacity(min)`:

- if $\text{current} \geq \text{min} \rightarrow$ unchanged
- if $\text{current} < \text{min} \rightarrow (2 \times \text{current}) + 2$
- if min is negative $\rightarrow$ no effect

`setLength()`: characters beyond the new length are lost.

CODE SNIPPET

```
StringBuffer sb3 = new StringBuffer("hello");
System.out.println(sb3.length());           // 5
System.out.println(sb3.capacity());          // 21

sb3.ensureCapacity(25);
System.out.println(sb3.capacity());          // 44

StringBuffer s1 = new StringBuffer("Sample test");
s1.setLength(5);
System.out.println(s1);                      // Sampl
```



charAt(), setCharAt() & getChars()

DEFINITION

charAt() reads a single character at an index.

setCharAt() replaces the character at an index.

getChars() copies a range of characters out into a target char array — all operate on a mutable StringBuffer.

SYNTAX

```
char charAt(int where) 0      'a'  
void setCharAt(int where, char ch)  
void getChars(srcStart, srcEnd, target[], targetStart)
```

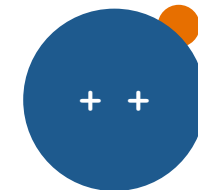
CODE SNIPPET

```
StringBuffer sb = new StringBuffer("Hello");  
System.out.println(sb.charAt(1));    // e  
sb.setCharAt(1, 'i');  
System.out.println(sb);              // Hillo  
  
StringBuffer s = new StringBuffer("This is nice");  
char t[] = new char[s.length()];  
s.getChars(2, s.length(), t, 0);
```

OUTPUT

```
e → Hillo  
t = "is is nice"
```

append() & insert()



append() — adds to the end

```
StringBuffer append(String str)
StringBuffer append(int num)
StringBuffer append(Object obj)
```

```
StringBuffer s = new StringBuffer("welcome");
s.append(" to ").append("CSE");
System.out.println(s);           // welcome to CSE
```

```
// chaining also works with mixed types:
StringBuffer sb = new StringBuffer("hai");
String r = sb.append("a = ").append(42).append("!").toString();
// r = "haia = 42!"
```

insert() — adds at an index

```
StringBuffer insert(int index, String str)
StringBuffer insert(int index, int num)
StringBuffer insert(int index, Object obj)
```

```
StringBuffer sb = new StringBuffer("I Java!");
sb.insert(2, "like ");
System.out.println(sb);
```

OUTPUT

```
welcome to CSE
haia = 42!
I like Java!
```



reverse(), delete() & replace()

reverse()

reverses the characters in place

```
sb=new StringBuffer("abcdef");  
sb.reverse();
```

"fedcba"

delete(startIndex, endIndex)

removes a range of characters, the endIndex which is exclusive

```
sb=new StringBuffer("This is a test.");  
sb.delete(4,7);
```

"This a test."

deleteCharAt(index)

removes one character

```
sb.deleteCharAt(0);
```

"his a test."

replace(startIndex, endIndex, newStr)

swaps a range for a new String, the endIndex which is exclusive

```
sb.replace(5,7,"was");
```

"This was a test."

ADDED IN JAVA 5

StringBuilder

DEFINITION

StringBuilder was added in Java 1.5. Its API is identical to StringBuffer's, but its methods are not synchronized — so it is not thread-safe, but offers faster performance in single-threaded code.

	StringBuffer	StringBuilder
Thread-safe	Yes (synchronized)	No
Speed	Slower	Faster
Since	JDK 1.0	JDK 1.5
API	Mutable, growable	Same as StringBuffer

Activity — Build a Mini Text Processor

Task

Write a Java program that accepts a sentence and:

1. Prints its length. ✓
2. Converts it to UPPERCASE and lowercase. ✓
3. Splits it into words using `split(" ")`. ✓
4. Counts how many times the letter 'a' appears using `indexOf()` in a loop. 
5. Reverses the sentence using a `StringBuilder`. 

```
//line.indexOf('a')
int c=0;
for(int i=0;i<line.length();) {
    i=line.indexOf('a',i);
    i=i+1;
    c=c+1;
}
System.out.println(c);

//StringBuilder - reverse
StringBuilder sb = new StringBuilder(line);
System.out.println(sb.reverse());
```

STARTER CODE

```
public class TextProcessor {
    public static void main(String[] a) {
        String line = "Java strings are powerful";

        System.out.println(line.length());
        System.out.println(line.toUpperCase());

        String[] words = line.split(" ");
        for (String w : words)
            System.out.println(w);

        // TODO: count occurrences of 'a'
        // TODO: reverse with StringBuilder
    }
}
```

Practice — Trace the Buffer

Q1. StringBuffer

```
StringBuffer s = new StringBuffer("123456789");  
s.delete(1,3).insert(1,"24");  
System.out.println(s);
```

Answer: 124456789

Q2. StringBuilder

```
StringBuilder s = new StringBuilder("123456789");  
s.delete(1,2).replace(1,5,"24");  
System.out.println(s);
```

Answer: 124789

Creative Thinking Questions

1. Why does Java make String immutable but allow StringBuffer to be mutable — what problem would arise if String itself could change in place?
2. Design a method that builds a dynamic email template like "Dear <Name>, your order <ID> ships on <Date>" using only String and StringBuilder — which would you choose, and why?
3. If you concatenate 10,000 Strings inside a loop with +, what happens internally, and how would StringBuilder change the outcome?
4. How would you use split(), trim() and equalsIgnoreCase() together to validate a CSV row of user input?

String Templates — Java's Journey

- 1 A "string template" means combining fixed text with variable values computed at runtime — the core idea behind dynamic text-processing output (emails, reports, JSON, SQL).
- 2 Java proposed a native interpolation syntax — String Templates — using `STR."Hello \{name}"` as JEP 430 (preview, JDK 21) and JEP 459 (second preview, JDK 22).
- 3 A third preview, JEP 465, was planned for JDK 23 — but after community feedback, Oracle concluded the design was unsuitable and withdrew it in March 2024. JDK 23 shipped with no string templates at all.
- 4 As of today, Java has no built-in string-interpolation syntax like Python's f-strings or JavaScript's template literals — the feature may return in a redesigned form, but there is no confirmed timeline.
- 5 In practice, Java developers achieve the same goal — dynamic string templates — using `String.format()`, `printf()`, text blocks with `.formatted()`, and `MessageFormat`, covered on the next slides.

String Template (Java 21 Preview)

 Java 

```
String name = "Alice";  
int age = 20;  
  
String message = STR."Name: \{name}, Age: \{age}";  
  
System.out.println(message);
```

Output



Name: Alice, Age: 20 

String.format() & printf()

DEFINITION

String.format() builds a new String from a pattern containing format specifiers (%s, %d, %f ...) filled in with values supplied afterward.

printf() applies the same formatting rules but prints directly instead of returning a String.

SYNTAX

```
static String format(String pattern,  
Object... args)  
PrintStream printf(String pattern, Object...  
args)
```

%s = String %d = integer %f / %.2f = decimal %n = newline

%s

CODE SNIPPET

```
String name = "Anu";  
int age = 21;  
  
String msg = String.format(  
    "Hello %s, you are %d years old.", name, age);  
System.out.println(msg);  
  
System.out.printf("Score: %.2f%n", 92.5);
```

OUTPUT

```
Hello Anu, you are 21 years old.  
Score: 92.50
```

Text Blocks & formatted()

DEFINITION

A text block (Java 13+, delimited by triple quotes """) holds multi-line literal text.

Calling `.formatted(args)` on it works exactly like `String.format()`, making text blocks ideal for multi-line templates such as emails, HTML, JSON, or SQL.

SYNTAX

```
String templateText = """
    multi-line text with %s
    """;
String result =
    templateText.formatted(args...)
```

CODE SNIPPET

```
String name = "Ravi", orderId = "ORD1042";

String template = """
    Dear %s,
    Your order %s has shipped.
    """;

String email = template.formatted(name, orderId);
System.out.println(email);
```

OUTPUT

```
Dear Ravi,
Your order ORD1042 has shipped.
```

MessageFormat — Locale-Aware Templates

DEFINITION

MessageFormat builds strings from a pattern using numbered placeholders {0}, {1}, ... instead of type-based specifiers.

It is especially useful for internationalized (i18n) messages that must adapt to different locales.

SYNTAX

```
static String format(String pattern,  
Object... args)
```

Placeholders refer to argument position, not type — the same argument can even be reused at multiple positions in the pattern.

CODE SNIPPET

```
String name = "Meera";  
int age = 21;  
  
String msg = MessageFormat.format(  
    "Hello {0}, you are {1} years old.", name, age);  
System.out.println(msg);
```

OUTPUT

```
Hello Meera, you are 21 years old.
```

Choosing the Right Templating Technique



Situation	Best Choice
One-off simple message	+ concatenation
Building text inside a loop	StringBuilder
Fixed pattern with placeholders	String.format()
Printing directly to console	printf()
Multi-line templates (email, HTML, SQL, JSON)	Text block + .formatted()
Locale / language-sensitive output	MessageFormat

Key takeaway: Java has no built-in interpolation syntax — `String.format()`/`.formatted()` is the standard way to generate dynamic string templates today.

Activity — Build a Dynamic Email Template

Task

Using a text block and `.formatted()`, write a program that generates a personalized order-confirmation message for 3 different customers stored in arrays:

1. Store name, orderId, and amount in parallel arrays.
2. Loop through the arrays.
3. For each customer, build the message from ONE shared text-block template.
4. Print all 3 generated messages.

Bonus: Add a `%.2f` amount and a currency symbol.

STARTER CODE

```
String[] names = {"Anu","Ravi","Meera"};
String[] orders = {"ORD101","ORD102","ORD103"};
double[] amounts = {499.0, 1250.5, 89.99};

String template = """
    Dear %s,
    Your order %s of $%.2f is confirmed.
    """;

for (int i = 0; i < names.length; i++) {
    String msg = template.formatted(
        names[i], orders[i], amounts[i]);
    System.out.println(msg);
}
```

Summary & Recap

1

String

- Immutable, fixed-length
- Constructors, length, concat
- Compare: equals, ==, compareTo
- Search: indexOf, lastIndexOf
- Modify: substring, replace, split

2

StringBuffer

- Mutable, growable
- Thread-safe (synchronized)
- append, insert, reverse
- delete, deleteCharAt, replace
- length vs capacity

3

StringBuilder

- Same API as StringBuffer
- Not synchronized
- Faster in single-threaded code
- Preferred for building dynamic templates
- Added in Java 5

Goal achieved: choose the right string-handling technique and generate dynamic, text-processing-ready String templates.

Activity – Classes and Objects

- Define an Employee class with data members: empId, name, and salary & function member to display employee's details.
- Use constructor to initialize the objects with user input
- **Bonus:** Add a method isHighSalary() that returns true if salary > 50000.
- Declare Employee[] employees = new Employee[4];
- Create 4 Employee objects using new.